

BYOS Business Email

V1 Prototype & Pre-Launch Engineering Roadmap

REVISED MASTER ROADMAP

STATUS: This revision incorporates the product, BYOS, frontend, infrastructure, email interoperability, organization-control, pricing, recovery, and cryptographic decisions made since the original roadmap. The crypto protocol is treated as a pre-implementation gate until the final cross-feature review is approved.

Product Vision

Build a full business-email platform that feels like a normal professional email service from the outside, while using customer-controlled persistent storage and a privacy-first cryptographic architecture underneath.

Customer provides persistent storage

BYOS provides email infrastructure + mailbox experience + routing + encryption + administration + interoperability

- Send and receive normal internet email with Gmail, Outlook, Yahoo, and other providers.
- Use custom domains, business mailboxes, aliases, contacts, folders, labels, rules, attachments, signatures, and scheduled sending.
- Use BYOS webmail and support standard desktop clients through SMTP/IMAP compatibility and the planned Bridge architecture.
- Support organization-managed and private mailbox modes.
- Keep persistent mailbox content encrypted in customer-controlled storage.
- Give every paid plan the same core product rather than artificial feature walls.

Core Principle

Normal email on the outside.

Customer-controlled storage underneath.

Privacy-aware encryption and organization control at the core.

REVISED AUGUST 2026

SECTION MAP

Roadmap Map

Use this page as the navigation map for the V1 build.

Phase	Sections	Outcome
A. Product + Security Foundation	0-4	Product rules, threat model, crypto gate, architecture, local lab
B. Mail Transport	5-8	Inbound/outbound SMTP, Rspamd, queues, trust and reputation controls
C. BYOS + Data	9-12	Storage abstraction, onboarding, management, cryptographic system
D. Mailbox Product	13-20	Organization model, auth/recovery, search, contacts, scheduled send, IMAP/Bridge, attachments
E. Reliability + Operations	21-24	Hardening, observability, backup/DR, exit/recovery tooling
F. Commercialization	25-30	Pricing, private beta, security review, production infra, scaling, implementation order

SCOPE: Post-V1 features such as native mobile apps, advanced calendar, full API, legal hold, advanced compliance controls, and browser-extension/native crypto should remain in a separate future backlog unless a real launch requirement moves them forward.

SECTION 0

Architecture, Threat Model & Product Rules

The mandatory design gate before production cryptography or production email infrastructure.

Do not treat architecture as documentation after the fact. The threat model, cryptographic model, feature/crypto compatibility matrix, and trust boundaries must be defined before the final implementation is locked.

Required deliverables

1. System Architecture Specification: every service, data flow, trust boundary, and API relationship.
2. Threat Model: attacks against web, API, mail, storage, identity, clients, devices, recovery, and abuse systems.
3. Cryptographic Specification: key generation, encryption, recovery, device enrollment, rotation, organization recovery, and private/managed mailbox behavior.
4. Data Classification: PUBLIC, OPERATIONAL METADATA, SENSITIVE METADATA, ENCRYPTED CUSTOMER DATA, SECRET/KEY MATERIAL.
5. Feature/Crypto Compatibility Matrix: every V1 feature checked for plaintext requirements, offline behavior, metadata exposure, and security impact.

Threat scenarios to model

- Compromised web server
- Compromised API
- Database leak
- Customer storage compromise
- Stolen credentials
- Lost/stolen device
- Compromised browser/client
- Malicious administrator or recovery principal
- SMTP credential abuse
- Storage outage
- Partial key rotation
- Scheduled-send compromise

Architecture rules that are now fixed

- Customer persistent mailbox storage is outside BYOS infrastructure.
- Go is the infrastructure/backend language; web stacks are separate.
- Next.js + React + TypeScript + Tailwind is the control plane/public management site.
- SolidJS + TypeScript + Tailwind is the interactive webmail client.
- Rust/WASM is the client cryptographic core; TypeScript orchestrates UI/API work.
- Postfix is the initial MTA and Rspamd is the initial filtering layer.
- Cloudflare is the web/edge layer; Linux VPS infrastructure handles mail services.
- No feature gating by plan. Plans primarily change mailbox capacity and legitimate usage limits.
- Detailed anti-spam/reputation rules will be designed when SMTP is implemented, but the outbound trust engine is mandatory architecture.

GATE: Implementation gate: no production crypto implementation until the final V1 crypto review resolves scheduled-send delivery, root-secret compromise, organization recovery credentials, live-mail rotation concurrency, HPKE wire format, and the browser trust boundary.

SECTION 1

Product & Organization Model

Organization hierarchy

```

Organization
|-- Owners
|-- Administrators
|-- Users
|-- Mailboxes
|-- Domains
|-- Aliases
|-- Storage connections
|-- Policies
`-- Billing
  
```

User and mailbox are separate concepts. A single employee can have multiple addresses that share a mailbox, while organization-level policies apply independently of mailbox-level encryption state.

Mailbox privacy modes

Mode	Meaning	Admin capability
Organization-managed	Business-controlled mailbox with authorized organizational recovery/access.	Can recover and act according to organization policy.
Private	Mailbox where the organization does not have automatic decryption capability.	Can block/archive/delete, but cannot automatically recover or export contents.

Employee termination

```

Terminate
-> disable authentication
-> disable SMTP submission
-> disable IMAP/Bridge access
-> preserve mailbox
-> apply retention policy
-> transfer / archive / delete according to mailbox mode and policy
  
```

POLICY: Private mailbox transfer/export is not an administrator capability in V1. The organization may request that the user export information before termination.

SECTION 2

Local Development Environment

Goal

Build the entire platform as a reproducible local laboratory before introducing production services or real customer mail.

```
Docker Desktop
|-- PostgreSQL
|-- Redis
|-- MinIO (S3-compatible)
|-- Postfix
|-- Rspamd
|-- Go services
`-- test utilities
```

Development toolchain

Layer	Technology	Purpose
Version control	Git + GitHub	Source control, review, history
Containers	Docker + Docker Compose	Reproducible local infrastructure
Backend	Go	Email/infrastructure/services
Frontend	Node.js + TypeScript	Web applications
Mailbox UI	SolidJS + Tailwind	Interactive webmail
Control plane	Next.js + React + Tailwind	Public/control/admin experience
Crypto	Rust + WebAssembly	Client cryptographic core
Mail	Postfix + Rspamd	SMTP transport and filtering

Reproducibility requirement

```
docker compose up
```

Expected: all local services boot from a clean machine and health checks pass without manual repair.

SECTION 3

Control Plane / Management Website

Stack

- Next.js
- React
- TypeScript
- Tailwind CSS
- Cloudflare web/edge deployment

Responsibilities

- Public website and SEO pages
- Pricing and documentation
- Authentication entry points
- Organizations and administrators
- User/mailbox provisioning
- Custom domain management
- Storage management
- Billing and subscriptions
- Security settings
- Account-level configuration

Suggested routes

```
/
/pricing
/security
/privacy
/docs
/blog
/login
/signup
/dashboard
/organization
/domains
/users
/billing
/storage
/security
```

BOUNDARY: The control plane is not the mailbox data processor. It manages business state and permissions while the mailbox client handles encrypted mailbox interaction.

SECTION 4

Webmail / Mailbox Client

Stack

- SolidJS
- TypeScript
- Tailwind CSS
- Vite / SolidStart where client/server rendering is useful
- Rust/WASM crypto core

Core V1 mailbox experience

Area	V1 capabilities
Mailbox	Inbox, Sent, Drafts, Archive, Spam, Trash
Organization	Folders, labels, threads, starred, read/unread
Message actions	Compose, reply, reply all, forward, signatures
Data	Attachments, inline images, contacts, aliases
Automation	Filters/rules, forwarding, auto-reply, scheduled send
Privacy	Local decryption, local search/index, security settings

Rendering model

Public/control pages -> SSR/SSG where appropriate
 Mailbox -> interactive client application + local crypto + local index

TRUST: The webmail application must assume the browser is trusted to execute the delivered client code in V1. A fully compromised origin can modify JavaScript and potentially extract client-held secrets. This is a documented residual risk, not something Rust/WASM alone eliminates.

SECTION 5

Email Infrastructure Foundation

Backend language

Go is the infrastructure/application language for the email platform. It remains separate from the frontend language choices.

Go responsibilities

- Mail routing
- Mailbox resolution
- API services where appropriate
- Storage connectors
- Workers
- Schedulers
- Encryption orchestration
- Queue processing
- Mail synchronization
- Administrative automation

Mail transfer components

Component	Role
Postfix	Established SMTP transfer and submission infrastructure. Do not write SMTP transport from scratch.
Rspamd	Spam/reputation/filtering layer, including SPF/DKIM/DMARC-related checks and policy hooks.
Redis	Queues, short-lived state, rate-limit counters, caching where appropriate.
PostgreSQL	Organization, mailbox, identity, metadata, audit, and job state.

SECTION 6

Inbound Email Pipeline

Gmail / Outlook / Yahoo / any internet mail server

|

v

DNS MX

|

v

BYOS SMTP Gateway

|

v

TLS / connection checks

|

v

Postfix

|

v

Rspamd

|

v

Mail routing / MIME parse

|

v

Encrypt + package

|

v

Customer-controlled storage

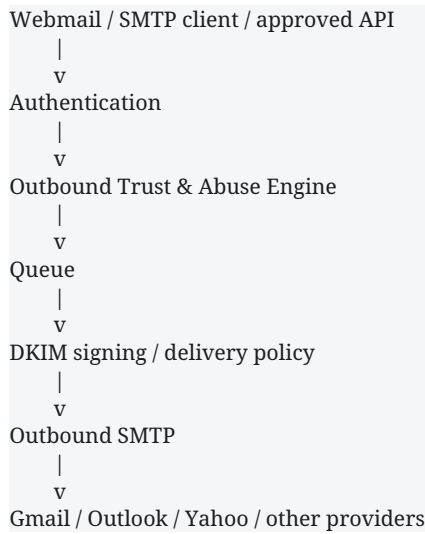
Requirements

- SMTP reception
- TLS
- MX handling
- Domain verification
- Mailbox routing
- MIME parsing
- Attachment handling
- Spam filtering
- Malware/security policy
- Bounce handling
- Retry behavior
- Message IDs
- Delivery state

PRIVACY: Inbound SMTP necessarily creates a transient plaintext processing window. The service must parse, route, and encrypt immediately, with no intentional plaintext persistence in database or object storage.

SECTION 7

Outbound Email Pipeline



Required standards

- SPF
- DKIM
- DMARC
- PTR / reverse DNS
- TLS

Single outbound control path

SMTP credentials must never bypass the outbound policy layer. Mail originating from webmail, Outlook, Apple Mail, Thunderbird, CRM software, or a custom application must pass through the same trust and abuse controls.

SECTION 8

Outbound Trust, Abuse & Reputation Engine

This is mandatory infrastructure, not a premium-plan feature. Detailed thresholds will be tuned during SMTP implementation and private beta.

Required controls

- Per-mailbox limits
- Organization limits
- Per-minute limits
- Hourly limits
- Rolling daily limits
- Recipients-per-message limits
- Burst detection
- New-recipient anomaly detection
- Account reputation
- Domain reputation
- IP reputation
- Automatic throttling
- Temporary suspension
- Abuse investigation
- Account recovery after compromise

Design principle

All plans -> complete email features

- |
- +-- safety limits -> protect shared infrastructure and IP reputation
- +-- established customers may earn higher trust ceilings
- `-- enterprise may receive custom limits after verification

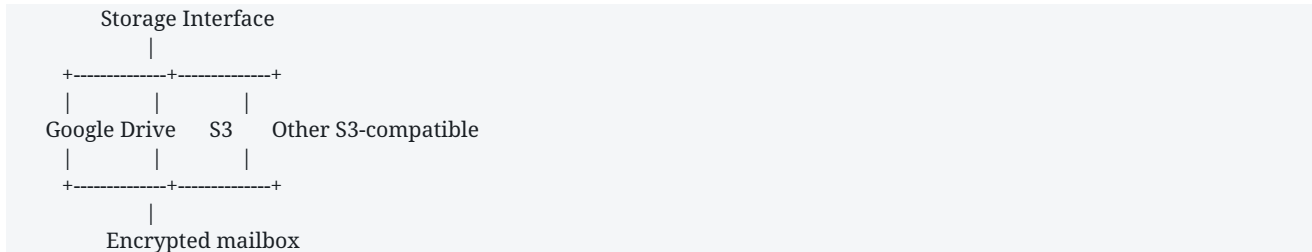
ANTI-ABUSE: A customer connecting external software through SMTP must not be able to turn a Solo account into an unrestricted relay. The service sells mailbox capability, not an unlimited bulk-mail cannon.

SECTION 9

BYOS Storage Architecture

Core principle

Persistent mailbox storage belongs to the customer. BYOS must use a storage abstraction so adding or replacing a provider does not require redesigning the mailbox system.



V1 storage priorities

Customer type	Default direction
Solo / small user	Google Drive is a convenient option; customer may also use S3-compatible storage.
Business / Team	Organization-level object storage is preferred; S3-compatible storage is the primary architecture path.
Technical users	Direct S3 or custom S3-compatible endpoint is supported where available.

Planned connectors

- Google Drive
- Amazon S3 / S3-compatible
- Google Cloud Storage
- Cloudflare R2
- Backblaze
- Wasabi
- MinIO
- Other compatible providers

PRICING PRINCIPLE: Plans do not restrict storage technology. A Solo customer with sophisticated S3 infrastructure is still a valid customer.

SECTION 10

BYOS Customer Experience

Solo onboarding

- Create account
 - > Create mailbox
 - > Connect storage
 - > Choose provider
 - > Authorize
 - > Generate keys
 - > Save recovery material
 - > Mailbox ready

Business onboarding

- Create organization
 - > Add domain
 - > Choose organization storage
 - > Connect storage
 - > Create users
 - > Create mailboxes
 - > Apply privacy policy
 - > Mailbox ready

Storage modes

Mode	Use case
Organization storage	One company storage source for many mailboxes; recommended for larger organizations.
Individual storage	A single mailbox owns its own storage connection; useful for Solo or technical users.
Hybrid	Central organization storage plus selected individual mailbox storage.

UX: Do not force 50 employees through 50 separate personal-drive authorizations. Organization storage should be the normal business path.

SECTION 11

Storage Management Layer

Customer dashboard

Storage

Provider: Amazon S3
Status: Healthy
Used: 182 GB
Available: 818 GB
Total: 1 TB
Last check: 2 minutes ago

Required functions

- Connect
- Disconnect
- Reconnect
- Test connection
- Capacity monitoring
- Health monitoring
- Authorization-failure detection
- Credential rotation
- Provider change
- Storage migration
- Failure reporting

Storage failure behavior

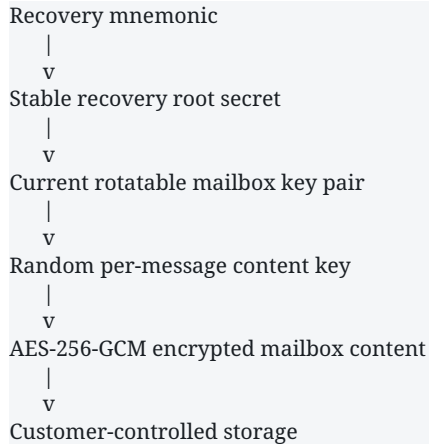
Storage outages must never silently become message loss. Inbound processing, queueing, retry, scheduled send, and mailbox access need explicit failure states and customer-visible status.

SECTION 12

Cryptographic System

The core privacy architecture. Detailed protocol implementation is gated on the final V4.1 cross-feature review.

Target model



Core rules

- Mailbox private keys are generated client-side and are never sent to the server unwrapped.
- A stable recovery root survives ordinary mailbox-key rotation.
- Per-message content keys are random and are wrapped for the current mailbox public key.
- Persistent mailbox content is encrypted before customer storage.
- Organization-managed mailboxes have an authorized organization recovery path.
- Private mailboxes do not have automatic administrator decryption.
- Authentication is separate from encryption keys.
- The server transiently processes inbound plaintext before encryption, but must not persist plaintext mailbox content.

Proposed protocol primitives

Layer	Primitive / role
Mailbox key	X25519 key pair, rotatable
Key wrapping	HPKE using a standardized RFC 9180 implementation; target suite DHKEM(X25519, HKDF-SHA256) + AES-256-GCM
Message content	Random 32-byte content key + AES-256-GCM
Derivation	HKDF-SHA256 for scoped sub-keys and recovery-root derivation
Admin passphrase	Memory-hard KDF such as Argon2id; exact parameters frozen before implementation
Client core	Rust/WASM crypto implementation with a stable JS API

Implementation gates still to resolve

- Scheduled Send without permanently exposing mailbox plaintext to the server
- Recovery-root rotation when root material is compromised
- Whether organization-managed employees receive independent mailbox recovery material
- Cryptographic revocation behavior for removed recovery principals

- Live-mail concurrency during long key rotations
- Canonical HPKE wire format
- Exact recovery mnemonic encoding/versioning
- Browser trust-boundary mitigations

SECTION 13

Organization-Managed vs Private Mailbox

Property	Organization-managed	Private
Normal user access	Yes	Yes
Device-based decryption	Yes	Yes
Organization recovery	Yes	No automatic recovery
Admin export	Yes, subject to policy	No automatic export
Transfer after employee leaves	Yes, via authorized recovery/re-key flow	No admin transfer without user cooperation
Business continuity	Strong	Dependent on user-controlled recovery

Mailbox creation UI

Mailbox privacy

(o) Organization-managed

Admin can access/recover according to organization policy.

(o) Private

Employee controls mailbox decryption; admin cannot automatically read it.

DESIGN: Privacy mode should be treated as a cryptographic state. In V1, changing between private and organization-managed should require an explicit cryptographic migration/re-key flow, not a silent database flag flip.

SECTION 14

Account Authentication

- Email + password
- Google login
- Microsoft login
- Optional two-step verification
- Passkeys as a later enhancement

Trust separation

Authentication credential

!=

Mailbox encryption key

!=

Recovery secret

Password reset and identity recovery restore account access. They do not automatically imply access to encrypted mailbox content.

SECTION 15

Account & Mailbox Recovery

Account recovery

Forgot login
-> verified recovery email / identity
-> restore account access

Mailbox recovery

PRIVATE
device OR recovery material
-> recover current mailbox keys

ORGANIZATION-MANAGED
authorized organization recovery principal
-> recover mailbox according to policy

Stable recovery root

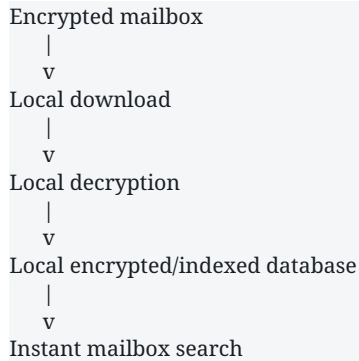
The recovery material derives a stable root secret rather than directly deriving the current mailbox private key. The current mailbox key can rotate while the recovery root remains stable.

SECURITY: Recovery-root rotation must exist as a distinct security operation. If the root secret or recovery credential is compromised, ordinary mailbox-key rotation is not sufficient by itself.

SECTION 16

Local Search & Search Privacy

Target architecture



V1 search model

For privacy-sensitive fields such as subject terms, the client can compute keyed search tokens locally and send only the token to the server. The server can return message IDs, while the client performs decryption locally.

Accepted V1 leakage

Leakage	V1 position
Token equality	Accepted; server may see that the same search token was used again.
Token value	Accepted; token does not directly reveal plaintext subject words, but repeated-token behavior remains visible.
Match-set size	Accepted; server can see result counts.
Access patterns	Accepted; server can observe which encrypted message objects are subsequently fetched.
Timing	Accepted; search time and API activity remain visible.

WORDING: Do not claim that the server cannot tell two searches use the same word/token. The correct statement is that the server does not receive the plaintext search term.

SECTION 17

Drafts & Contacts

Drafts

- Encrypted storage
- Autosave
- Multi-device synchronization
- Attachment handling
- Recovery

Contacts

- Customer-controlled storage as encrypted data
- Client-side management
- Separation between private address-book data and operational sender/recipient data

PRIVACY: All features that normally feel "small" in an email client are treated as real mailbox data when they contain customer content.

SECTION 18

Scheduled Sending

REQUIREMENT: Scheduled Send is V1 and must work while the user device is offline.

```
Compose
|
v
Schedule for future time
|
v
Persist safely
|
v
Scheduler
|
v
Outbound Trust & Abuse Engine
|
v
SMTP delivery
```

Critical cryptographic design question

The scheduled message must remain protected at rest, yet the server must be able to deliver it after the user device is offline. This means Scheduled Send cannot be treated as a normal client-only feature.

Pre-implementation decision required

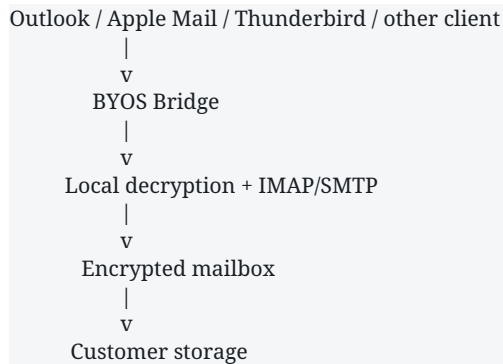
- Define a narrowly scoped server delivery capability or an alternative mechanism that does not grant general mailbox decryption.
- Define cancellation and editing.
- Define attachment handling.
- Define storage outage behavior.
- Define exact server visibility at send time.
- Run the same outbound abuse controls used for ordinary mail.

SECTION 19

IMAP / SMTP Client Compatibility

Requirement

Customers must be able to use normal external mail clients. The architecture must not trap users inside the webmail UI.



Scope

- Internet SMTP interoperability
- Client submission support
- IMAP-compatible mailbox access
- Bridge protocol design
- Credential and session management
- Local decryption

ARCHITECTURE: Bridge implementation can follow the stable mailbox/encryption protocol, but Bridge compatibility is an architectural V1 requirement and must not be accidentally designed out.

SECTION 20

Attachment System

V1 limit

Maximum total message size: 40 MB

- Upload
- Download
- Inline images
- MIME parsing
- Encrypted storage
- Attachment metadata
- Download authorization
- Cleanup
- Failure recovery

Treat 40 MB as the complete message payload budget in V1 rather than assuming every external provider accepts a 40 MB individual attachment. A future secure file-sharing feature can handle larger transfers.

SECTION 21

Security & Application Hardening

Application security

- Secure cookies
- CSRF protection
- XSS defenses
- SQL injection protection
- SSRF protection
- Strict authorization
- Rate limiting
- Secure headers
- Dependency scanning
- Secret management
- Session management

Infrastructure security

- Firewall
- Hardened SSH
- Restricted service exposure
- Service isolation
- Security updates
- Backups
- Restore testing
- Secrets outside source control

Key security

- Key rotation
- Recovery-root rotation
- Device revocation
- Recovery protection
- Storage credential rotation
- Least-privilege storage access

Browser trust

V1 acknowledges that a fully compromised server delivering malicious JavaScript can potentially steal secrets used by the web client. Mitigations include strong deployment controls, reproducible builds, code auditing, strict CSP where appropriate, key verification flows, and a future separately trusted browser extension/native client.

SECTION 22

Monitoring & Observability

Tool / layer	Purpose
Prometheus	Metrics
Grafana	Visualization / operational dashboards
Sentry	Application errors and diagnostics
External uptime monitoring	Availability from outside the infrastructure

Monitor

- SMTP gateways
- Mail queues
- Workers
- PostgreSQL
- Redis
- Cloudflare edge/services
- Storage connectors
- Authentication
- Outbound delivery
- Bounce rates
- Abuse events
- CPU / RAM / network / disk

OPERATIONS: Monitoring must include customer-impacting storage and delivery failures, not just server health.

SECTION 23

Backup & Disaster Recovery

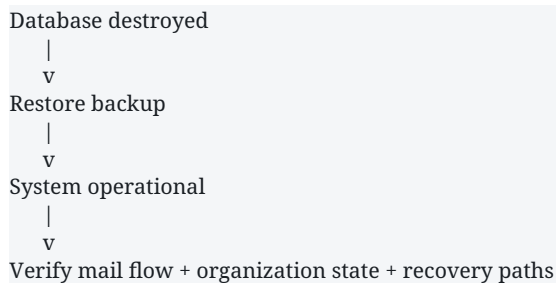
Back up

- Control-plane database
- Service configuration
- Operational metadata
- Cryptographic metadata that is intentionally recoverable
- Infrastructure configuration

Do not build

- A giant BYOS-owned plaintext mailbox archive merely to simplify backups.

Required restore test



```
graph TD; A[Database destroyed] --> B[Restore backup]; B --> C[System operational]; C --> D[Verify mail flow + organization state + recovery paths]
```

Database destroyed
|
v
Restore backup
|
v
System operational
|
v
Verify mail flow + organization state + recovery paths

SECTION 24

Independent Exit / Recovery Tool

A major trust feature is the ability for a customer to leave without being permanently dependent on the SaaS platform.



Requirements

- Works without a live SaaS dependency
- Exports mailbox data
- Respects encryption and recovery rules
- Supports organization-managed archive/recovery use cases
- Has explicit documentation and test vectors

TRUST: The export path is part of the product promise, not merely an internal engineering utility.

SECTION 25

Billing & Pricing

Pricing philosophy

Every paid plan receives the same complete core BYOS email product. Customers pay primarily for mailbox capacity and scale, not for artificial feature unlocks.

Plan	Mailboxes	Monthly	Annual
Solo	1	\$3	\$31
Starter	5	\$14	\$143
Business	10	\$26	\$265
Team	25	\$60	\$612
Business+	50	\$110	\$1,122
Enterprise	50+	Custom	Custom

All plans include the core product

- Custom domain
- SMTP
- IMAP/Bridge architecture
- BYOS storage connectors
- Encryption
- Search
- Contacts
- Aliases
- Rules
- Scheduled Send
- 2FA
- Google/Microsoft sign-in
- Core organization controls where applicable

What can vary

- Mailbox count
- Legitimate usage ceilings
- Organization capacity
- Enterprise support/contract terms
- Anti-abuse limits required to protect shared infrastructure

PRICING: S3 is not a premium feature. A developer can use Solo and connect their own S3-compatible storage.

SECTION 26

Private Beta

Staged rollout

5-10 trusted testers
-> 10-25 businesses
-> 25-50 businesses
-> 100 businesses

Measure at every stage

- Delivery reliability
- Spam placement
- Storage failures
- Encryption failures
- Recovery success
- Onboarding completion
- Account termination flows
- Scheduled send
- SMTP/IMAP client compatibility
- Support volume
- Infrastructure load

ROLLOUT: Do not jump from a working prototype to thousands of customers. Email reputation, storage edge cases, recovery, and client interoperability need real-world observation.

SECTION 27

Security Review & Launch Gate

Independent review focus

- Cryptography
- Key management
- Authentication
- Storage authorization
- SMTP security
- Tenant isolation
- Administrator access
- Recovery
- Web-client trust boundary
- Scheduled-send security
- Device revocation
- Root-rotation behavior

Launch sequence

Internal review

- > findings
- > fixes
- > retest
- > independent security review
- > private beta
- > remediation
- > production launch

SECURITY: A bug bounty is a later layer, not a substitute for a security review and controlled private beta.

SECTION 28

Production Infrastructure

Web / edge

```
Cloudflare
|-- DNS
|-- TLS
|-- WAF / edge controls
|-- static assets
`-- web application delivery
```

Email / backend

```
Linux VPS
|-- Postfix
|-- Rspamd
|-- Go services
|-- queues / workers
`-- supporting mail infrastructure
```

Data

```
PostgreSQL
Redis

Customer mailbox storage
Google Drive / S3 / S3-compatible
```

Production principles

- Start small
- Keep components replaceable
- Use infrastructure-as-code as complexity grows
- Keep secrets outside source control
- Separate public web traffic from mail infrastructure
- Do not expose internal services directly to the internet unless required

SECTION 29

Scaling Model

Scale	Likely evolution
0-1,000 mailboxes	Small production cluster, strong monitoring, conservative outbound reputation policy.
1,000-10,000	Additional SMTP gateways, worker capacity, queue scaling, database capacity, reputation segmentation.
10,000-50,000	Multiple SMTP gateways, queue/worker clusters, stronger database strategy, more advanced delivery isolation.
50,000+	High availability, database replicas, multiple regions where justified, advanced IP pools, dedicated enterprise infrastructure.

SCALE: Scaling decisions should be driven by measured bottlenecks, not by premature complexity. BYOS removes much of the mailbox-object storage burden, but bandwidth, metadata, queueing, indexing, CPU, and outbound reputation still scale with usage.

SECTION 30

Implementation Order - Where We Start

This is the operational sequence to hand to OpenCode and use as the project checklist.

Step 0 - Freeze the architecture gate

- Complete the final security/crypto compatibility review.
- Resolve Scheduled Send, root rotation, organization recovery credentials, recovery-principal revocation, live-mail key rotation concurrency, HPKE wire format, and browser trust assumptions.

Step 1 - Build the local laboratory

Docker
PostgreSQL
Redis
MinIO
Postfix
Rspamd
Go services
Test utilities

Step 2 - Implement the final Rust/WASM crypto core

- Use an established HPKE implementation; do not hand-roll HPKE.
- Implement recovery-root logic, mailbox keys, message-key wrapping, AES-GCM, HKDF, and exact wire-format serialization.
- Validate against official test vectors and project-specific test vectors.

Step 3 - Complete the inbound vertical slice

External test sender
-> SMTP
-> Postfix
-> Rspamd
-> Go router
-> encrypt
-> MinIO
-> webmail client
-> local decrypt
-> display message

MILESTONE: Success condition: the stored mailbox object is unreadable plaintext, while the authorized client can decrypt and display the message.

Step 4 - Implement outbound mail

Webmail / SMTP client
-> authentication
-> outbound trust engine
-> queue
-> Postfix
-> external mailbox

Step 5 - Implement BYOS

- Start with MinIO locally.
- Add S3 / S3-compatible storage.
- Add Google Drive.

- Keep a provider-neutral storage interface.

Step 6 - Build the web applications

Control Plane: Next.js + React + Tailwind
 Mail Client: SolidJS + TypeScript + Tailwind
 Crypto: Rust/WASM shared client module

Step 7 - Build organizations & administration

- Organizations
- Owners/admins
- Users
- Mailboxes
- Domains
- Aliases
- Privacy modes
- Storage management
- Termination/recovery
- Security events

Step 8 - Complete core mailbox features

- Drafts
- Contacts
- Search
- Attachments
- Labels
- Filters
- Forwarding
- Auto-reply
- Scheduled Send
- Signatures

Step 9 - Implement IMAP/SMTP Bridge compatibility

Do this after the mailbox/encryption protocol is stable, while preserving the compatibility requirement from the beginning.

Step 10 - Reliability, hardening, and adversarial testing

- Monitoring
- Backups
- Disaster recovery
- Crypto vectors
- Storage failure tests
- Rotation interruption tests
- Concurrent incoming-mail tests
- Recovery tests
- Abuse tests
- Tenant isolation tests

Step 11 - Private beta

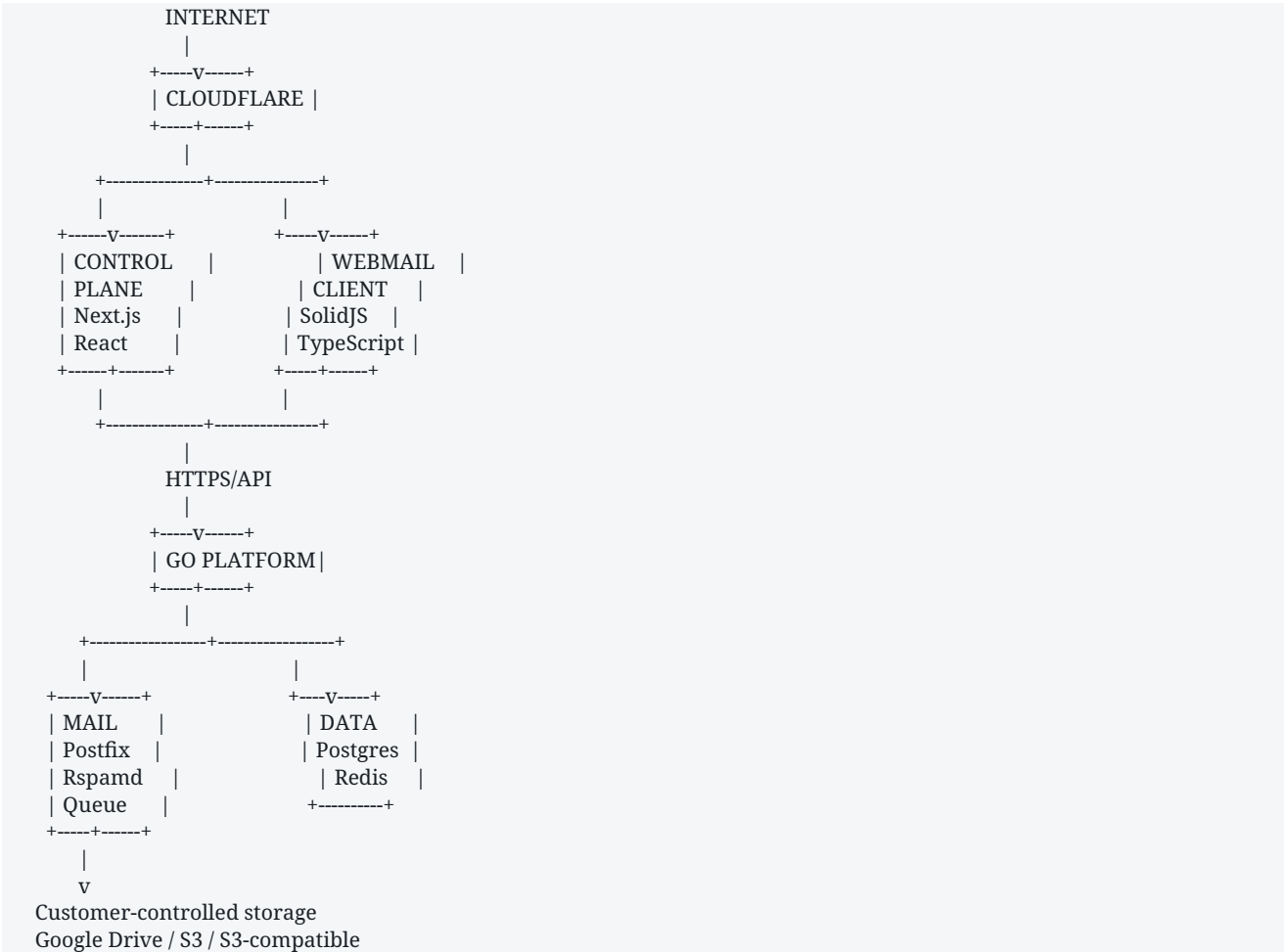
5 -> 10 -> 25 -> 50 -> 100 businesses

Step 12 - Public launch

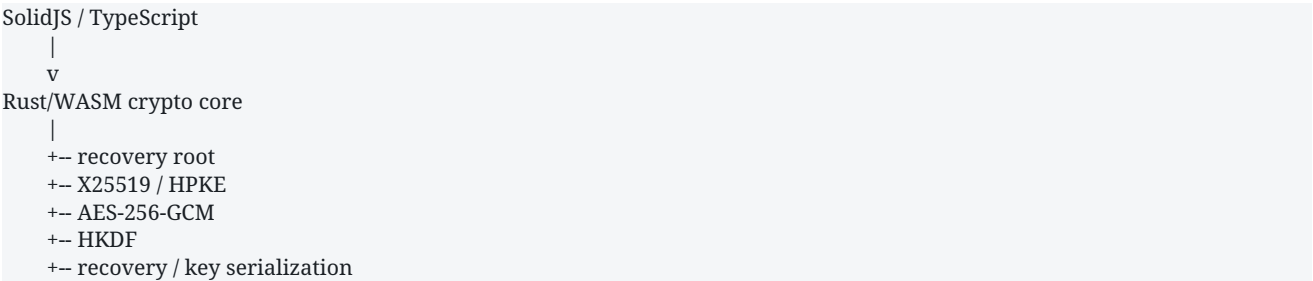
- Core product works
- Email interoperability works
- BYOS works
- Encryption/recovery validated
- Security review complete
- Monitoring and backups validated
- Abuse controls validated
- Private beta stable

SECTION A

Appendix A - Current Architecture Snapshot



Client crypto boundary



SECTION B

Appendix B - Locked Product Decisions

Decision	Current choice
Product model	Full business email + customer-controlled persistent storage.
BYOS	Organization, individual, and hybrid storage supported.
Pricing	No feature gating; capacity-based tiers.
Frontend split	Next.js/React control plane + SolidJS mailbox.
Infrastructure language	Go.
Client crypto	Rust/WASM.
Mail transfer	Postfix.
Filtering	Rspamd.
Web edge	Cloudflare.
Mailbox storage	Customer-controlled storage.
Attachment limit	40 MB total message size in V1.
Mailbox modes	Organization-managed + Private.
Scheduled Send	V1; works while client is offline; crypto design gate required.
SMTP/IMAP	Standard interoperability required; Bridge architecture planned.
Search	Client-oriented privacy model with acknowledged token/access-pattern leakage in V1.
Recovery	Separate account recovery from mailbox recovery; stable recovery root model.
Organization recovery	Multiple authorized recovery principals; exact revocation behavior must be explicit.
Outbound safety	Unified trust/reputation engine for webmail, SMTP clients, and approved APIs.
Exit	Independent mailbox export/recovery capability.

SECTION C

Appendix C - V1 Acceptance Checklist

Gate	Pass condition
Architecture	Threat model, crypto spec, feature compatibility matrix approved.
Crypto	Test vectors and recovery-after-rotation pass; no global mailbox decryption key exists.
Inbound	External message received, filtered, encrypted, stored, retrieved, and decrypted by authorized client.
Outbound	External delivery works with SPF/DKIM/DMARC/PTR/TLS and trust controls.
BYOS	Customer storage can be connected, health-checked, recovered, and changed.
Organization	Admins/users/domains/mailboxes/privacy modes/termination behave correctly.
Webmail	Core mailbox functions work with local decrypt/search architecture.
Client compatibility	SMTP/IMAP Bridge architecture is stable and protocol-compatible.
Reliability	Backups restore; storage/queue/server failures have tested behavior.
Security	Independent review complete; findings fixed and retested.
Beta	Staged private beta produces acceptable delivery, reliability, recovery, and support metrics.
Launch	All launch conditions are met before public signup is enabled.

SECTION D

Appendix D - Post-V1 Future Backlog

- Browser-extension crypto boundary for stronger protection against compromised web origins.
- Native desktop client and stronger offline support.
- Native mobile applications and push notifications.
- Mailbox migration/import tooling from existing providers.
- Shared mailbox/delegation enhancements.
- Passkeys.
- Calendar integration.
- API and deeper business integrations.
- Retention and advanced archival/legal-hold features.
- Additional storage providers and advanced data-residency controls.
- Advanced enterprise compliance features.
- Private-information-retrieval or stronger search-privacy techniques if justified by demand.

Final V1 Position

The V1 product is a complete business-email service with normal internet interoperability, customer-controlled persistent storage, privacy-aware mailbox encryption, organization administration, recovery, scheduled sending, and a path to standard desktop clients. The architecture is intentionally split so each major concern has a clear home: Next.js/React for the control plane, SolidJS for the mailbox, Rust/WASM for client cryptography, and Go/Postfix/Rspamd for the mail platform.

PAY FOR CAPACITY.
CHOOSE YOUR STORAGE.
USE NORMAL EMAIL.
KEEP CONTROL OF YOUR DATA.

CANONICAL: This roadmap is the canonical V1 project plan. Detailed cryptographic protocol files may expand or refine the crypto sections, but product-wide decisions recorded here should not be silently changed during implementation.